ΕΛΛΗΝΙΚΗ ΔΗΜΟΚΡΑΤΙΑ
Εθνικόν και Καποδιστριακόν Πανεπιστήμιον Αθηνών
ΙΔΡΥΘΕΝ ΤΟ 1837

Παράλληλα Συστήματα - Εργασία 1

Ιωάννης Γρηπιώτης (sdi2100028)
Θεμιστοκλής Παπαθεοφάνους (sdi2100227)

ΠΕΡΙΛΗΨΗ

Όλες οι ασκήσεις εκτελέστηκαν πάνω στο σύστημα linux02.di.uoa.gr του Πανεπιστημίου Αθηνών, το οποίο διαθέτει:

- επεξεργαστή Intel Core i7-6700 4c/8t @ 3.40GHz,
- 16GB μνήμη RAM,
- λειτουργικό σύστημα Ubuntu 20.04.6 LTS,
- έκδοση πυρήνα 5.4.0-216-generic,
- έκδοση μεταγλωττιστή GCC 9.4.0

Κάθε άσκηση εκτελέστηκε 10 φορές με την μέση τιμή του χρόνου εκτέλεσης να υπολογίζεται από το ίδιο το εκτελέσιμο.

Κάθε άσκηση εκτελείται από το εκτελέσιμο ως εξής:

```
./scripts/run_<exercise_name>.sh
```

όπου το `<exercise_number>` είναι ο αριθμός της άσκησης που πρόκειται να εκτελεστεί και τα `FLAGS` είναι οι παράμετροι (οι οποίες έχουν το πρόθημα `-f`) που δίνονται στην εκάστοτε άσκηση.

ΑΣΚΗΣΗ 1.1

Περιγραφή προβλήματος

Γινόμενο 2 πλήρων πολυωνύμων μεγάλου βαθμού. Συγκεκριμένα έγιναν πειράματα πάνω σε 10^3, 10^4, 10^5.

Περιγραφή λύσης

Δημιουργία πολυωνύμων

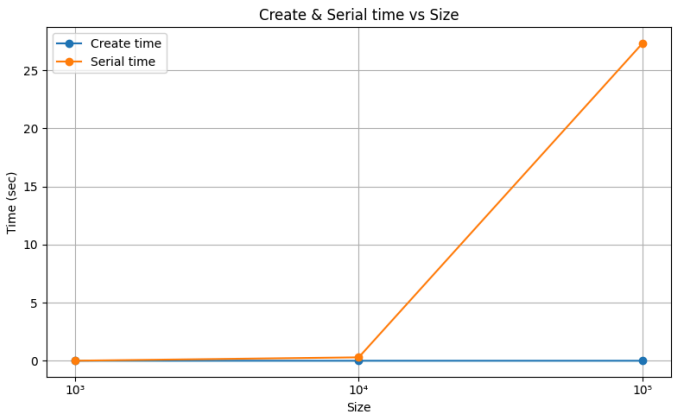
Χρησιμοποιήθηκαν threads, όσα ήταν στη διάθεση, προκειμένου να γίνει όσο πιο γρήγορα γίνεται η αρχικοποίηση. (**κέρδος** 1 τάξη μεγέθους σε sec)

Υλοποίηση γινομένου

Χωρίστηκε το 1 από τα 2 πολυώνυμα σε ίσα μέρη ανάλογα με τον αριθμό των threads και έπειτα το κάθε thread έκανε το γινόμενο των 2 πολυωνύμων σε έναν δικό του μοναδικό χώρο. Στο τέλος όλα τα threads ένωσαν τα αποτελέσματά τους σε έναν τελικό ενιαίο χώρο.

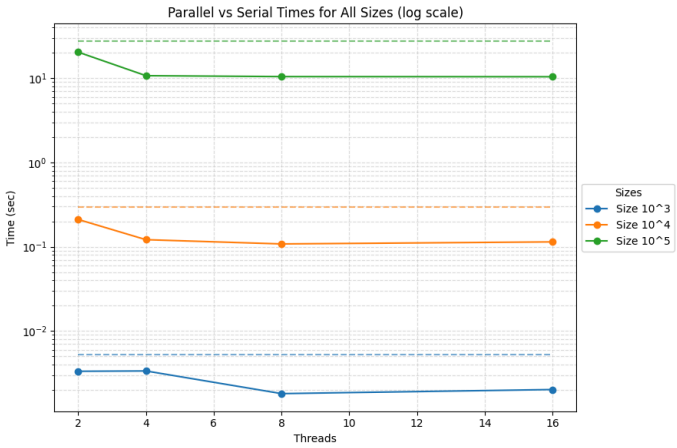
Αποτελέσματα

Τα παρακάτω αποτελέσματα δεν επηρεάζονται από τον αριθμό των threads



Οι χρόνοι είναι σε [sec]

Size	Create time	Serial time
1000	0.000776	0.005184
10000	0.000994	0.291378
100000	0.001097	27.346316



10^3 Βαθμός πολυωνύμων. Σειριακός χρόνος 0.005184 [sec] (baseline)

Threads	Parallel time[sec]	Speed up[times]	Efficiency
2	0.003338	1.56x	0.78
4	0.003367	1.55x	0.39
8	0.001814	2.88x	0.36
16	0.002027	2.57x	0.16

10^4 Βαθμός πολυωνύμων. Σειριακός χρόνος 0.291378 [sec] (baseline)

Threads	Parallel time[sec]	Speed up[times]	Efficiency
2	0.21085	1.4x	0.7
4	0.121368	2.44x	0.61
8	0.108046	2.74x	0.34
16	0.114285	2.59x	0.16

10^5 Βαθμός πολυωνύμων. Σειριακός χρόνος 27.346316 [sec] (baseline)

Threads	Parallel time[sec]	Speed up[times]	Efficiency
2	20.341295	1.34x	0.67
4	10.692184	2.56x	0.64
8	10.428751	2.62x	0.33
16	10.390952	2.63x	0.16

Συμπέρασμα-Παρατηρήσεις

Δημιουργία και Σειριακή εκτέλεση

Η αύξηση του χρόνου είναι αναμενόμενη καθώς αυξάνεται το μέγεθος, μιας και πάντα χρησιμοποιείται σταθερός αριθμός από threads ή και καθόλου, αντίστοιχα.

Παράλληλη εκτέλεση γινομένου

Παρατηρείται ότι ασχέτως του βαθμού των πολυωνύμων υπάρχει μια συνεχής αύξηση του χρόνου υπολογισμού. Εκτός από τα 16 threads τα οποία σχεδόν πάντα μειώνουν τον χρόνο. Αυτό το φαινόμενο παρατηρείται εξαιτίας της φύσης του hardware μιας και τα πειράματα έγιναν σε 8πύρηνο σύστημα.

Εξαιτίας αυτού δεν μπορεί να υπάρξει όντως παράλληλη εκτέλεση και των 16 threads. Θεωρητικά σε 16πύρηνο σύστημα θα παρατηρούνταν περισσότερο speed up. Ο λόγος που υπάρχει συνεχής αύξηση είναι λόγω του ότι "σπάει" η δουλειά σε μικρότερα παράλληλα κομμάτια.

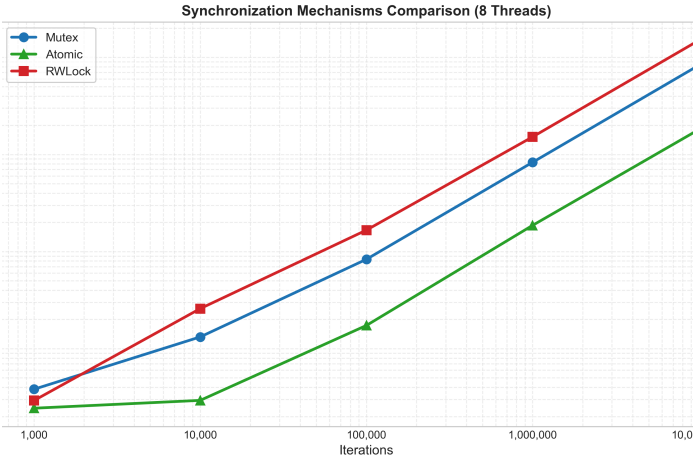
Επομένως το speed up είναι αρκετά αναμενόμενο, αν και φραγμένο μιας και το κύριο πρόβλημα είναι ότι υπάρχουν αρκετά L1 cache miss λόγω του μεγέθους των πολυωνύμων με αποτέλεσμα να μην μπορούν να κρατηθούν για όλη τη διάρκεια ολόκληρο το δεύτερο πολυώνυμο αλλά μόνο κομμάτια του τα οποία εναλλάσσονται συνεχώς.

Συγχρονισμός

Λόγω της προσέγγισης που ακολουθήθηκε, κάθε thread αποθηκεύει τα αποτελέσματά του σε έναν δικό του χώρο στο heap. Αυτό επιτρέπει τον υπολογισμό όντως παράλληλα μερών των λύσεων. Στο τέλος υπάρχει ένα **Coarse-grained locking** προκειμένου σειριακά και **σωστά-συγχρονισμένα** να μπορεί το κάθε thread να προσθέσει στο τελικό αποτέλεσμα, κοινός χώρος για όλα στο heap, το δικό του μέρος λύσης.

ΑΣΚΗΣΗ 1.2

Άσκηση 2: Αύξηση Κοινόχρηστης Μεταβλητής



Πολλαπλά νήματα αυξάνουν μία κοινόχρηστη μεταβλητή. Υλοποιήθηκαν τρεις μηχανισμοί συγχρονισμού: mutex, rwlock και atomic operations σε write-only workload. Τα benchmarks εκτελέστηκαν με 2, 4 και 8 threads για $N \in \{10^3, 10^4, 10^5, 10^6, 10^7\}$ iterations.

Ανάλυση

Atomic operations: Σαφής νικητής. Τα atomics υπερέχουν σε όλα τα μεγέθη workload και thread counts. Για $N = 10^7$ με 8 threads: atomics 0.181s vs mutex 0.830s vs rwlock 1.498s. Τα atomics είναι **4.6× ταχύτερα** από το mutex και **8.3× ταχύτερα** από το rwlock. Η υπεροχή είναι ακόμη μεγαλύτερη σε μικρά workloads ($N = 10^3$: atomics 0.00024s vs mutex 0.00040s = 1.67× ταχύτερα).

Thread scalability - Atomics: Τα atomics κλιμακώνονται εξαιρετικά. Για $N = 10^7$: 2t=0.180s, 4t=0.178s, 8t=0.181s - σχεδόν μηδενική υποβάθμιση με αύξηση threads. Επιτυγχάνουν 55 Mops/sec ανεξαρτήτως thread count, αποδεικνύοντας ελάχιστο cache coherence overhead.

Thread scalability - Mutex: Το mutex υποβαθμίζεται σοβαρά με περισσότερα threads. Για $N = 10^7$: 2t=0.386s → 4t=0.397s → 8t=0.830s. Με 8 threads είναι **2.1× πιο αργό** από 2 threads, λόγω kernel context switches και lock contention. Για μικρά workloads ($N = 10^3$) είναι ανταγωνιστικό (0.30ms με 2t), αλλά καταρρέει με περισσότερα threads (0.40ms με 8t).

Thread scalability - Rwlock: Το rwlock έχει τη χειρότερη κλιμάκωση. Για $N = 10^7$: 2t=0.869s → 4t=1.010s → 8t=1.498s. Με 8 threads είναι **1.72× πιο αργό** από 2 threads. Σε μικρότερα workloads η υποβάθμιση είναι ακόμη χειρότερη ($N = 10^5$: 2t=8.8ms → 8t=16.6ms = 1.89× πιο αργό).

Αιτίες υπεροχής atomics:

- Lock-free: μηδενικό kernel overhead, χωρίς context switches
- Hardware-level synchronization: LOCK prefix στο x86 αντί για syscalls
- Minimal cache footprint: single cache line vs πολύπλοκες lock structures
- Αποφυγή priority inversion και scheduler overhead

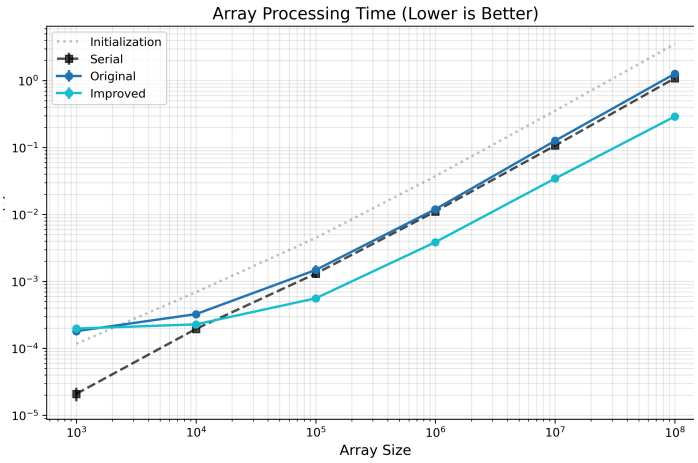
Γιατί το rwlock αποτυγχάνει:

- Write-only workload → μηδενικός παραλληλισμός (όλα τα threads αποκλείονται)
- Πολύπλοκη εσωτερική δομή (reader/writer queues) χωρίς όφελος
- Overhead 4-6 atomic operations ανά lock/unlock vs 2 για mutex
- Μεγαλύτερη δομή → περισσότερο cache coherence traffic

Συμπέρασμα

Για write-only workloads: **atomics >> mutex >> rwlock**. Τα atomic operations είναι η βέλτιστη επιλογή σε όλα τα σενάρια, προσφέροντας 4-8× καλύτερη απόδοση και εξαιρετική scalability. Το mutex υποβαθμίζεται με αύξηση threads αλλά παραμένει χρησιμοποιήσιμο. Το rwlock δεν δικαιολογείται ποτέ σε write-heavy scenarios - σχεδιάστηκε αποκλειστικά για read-heavy workloads με concurrent readers.

ΑΣΚΗΣΗ 1.3



Size	Serial (s)	Orig (s)	Imp (s)	Spd(Orig)	Spd(Imp)
1,000	0.000021	0.000181	0.000198	0.12x	0.11x
10,000	0.000196	0.000325	0.000229	0.60x	0.86x
100,000	0.001309	0.001484	0.000560	0.88x	2.34x
1,000,000	0.011119	0.012005	0.003869	0.93x	2.87x
10,000,000	0.107060	0.126360	0.034498	0.85x	3.10x
100,000,000	1.086645	1.265788	0.291205	0.86x	3.73x

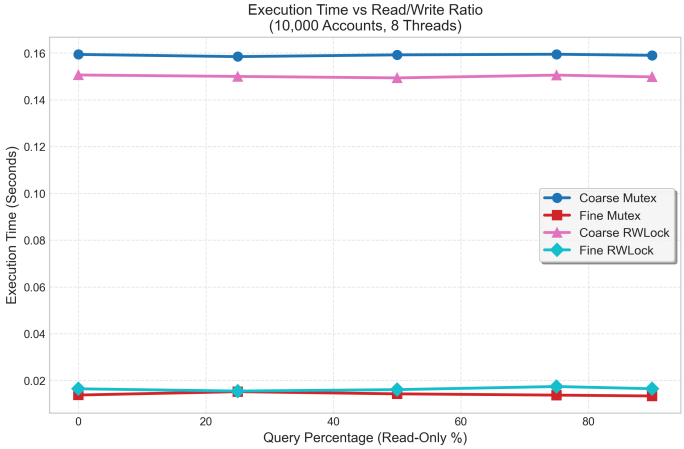
Η αρχική παράλληλη υλοποίηση είναι πιο αργή από τον σειριακό αλγόριθμο για $N < 10^5$ λόγω του overhead των threads. Για μεγαλύτερες τιμές N φαίνεται ότι η παράλληλη υλοποίηση του αλγορίθμου εξακολουθεί να είναι αργότερη της σειριακής , λόγω του φαινομένου της ψευδούς κοινοχρησίας και του ping pong effect. Αυτό συμβαίνει διότι τα fields του struct είναι εντός cache line το οποίο οδηγεί στην παρουσίαση των φαινομένων που αναφέρθηκαν προηγουμένως.

Για να λυθούν τα φαινόμενα που παρουσιάζονται και αποτρέπουν την παρατήρηση της επιθυμητής επιτάχυνσης, τροποποιήθηκε η δομή προσθέτοντας padding ανάμεσα στο κάθε field ώστε να αναγκαστούν να βρεθούν σε διαφορετικά cache lines. Με την τροποποίηση αυτή παρατηρήθηκε άμεση βελτίωση έπειτα από το cross over σημείο της παράλληλης με τη σειριακή υλοποίηση ξεκινώντας από 2.34x για $N = 10^5$, φτάνοντας το 3.73x για $N = 10^8$.

Ιωάννης Γρηπιώτης (sdi2100028)
Θεμιστοκλής Παπαθεοφάνους (sdi2100227)

ΑΣΚΗΣΗ 1.4

Άσκηση 4: Τραπεζικές Συναλλαγές



Προσέγγιση	0% Query	25% Query	50% Query	90% Query
Coarse Mutex	159.4 ms	158.5 ms	159.2 ms	159.3 ms
Fine Mutex	13.8 ms	15.2 ms	14.3 ms	13.3 ms
Coarse RWLock	149.7 ms	150.0 ms	149.3 ms	149.8 ms
Fine RWLock	16.4 ms	15.5 ms	16.1 ms	16.5 ms

Παράμετροι: 10K accounts, 8 threads, 75K συναλλαγές

Προσομοίωση τράπεζας με transfers (write) και queries (read) σε κοινόχρηστο πίνακα λογαριασμών. Υλοποιήθηκαν 4 σχήματα συγχρονισμού: coarse/fine-grained με mutex/rwlock.

Coarse vs Fine-Grained

Το fine-grained είναι 11.5x ταχύτερο από το coarse-grained (13.8 ms vs 159.4 ms για 0% queries). Με 10K accounts και τυχαίες συναλλαγές, η πιθανότητα δύο threads να χρειαστούν τον ίδιο λογαριασμό είναι 0.08%. Το coarse-grained κλειδώνει όλους τους λογαριασμούς σε κάθε συναλλαγή, ενώ το fine-grained κλειδώνει μόνο τους 2 που χρειάζεται. Έτσι στο coarse-grained τα threads εκτελούνται σειριακά, ενώ στο fine-grained τρέχουν παράλληλα χωρίς contention.

Accounts	Coarse Mutex	Fine Mutex	Speedup
100	3.5 ms	0.7 ms	5.0×
1,000	15.7 ms	3.1 ms	5.1×
10,000	159.3 ms	13.8 ms	11.5×

Το όφελος του fine-grained αυξάνεται με περισσότερα accounts (5.0x → 11.5 x) διότι μειώνεται η πιθανότητα contention (10% για 100 accounts → 0.08% για 10K accounts).

Mutex vs RWLock

Στο fine-grained το mutex είναι ελαφρώς ταχύτερο (13.3-15.2 ms vs 15.5-16.5 ms, 10-15% βελτίωση). Στο coarse-grained το rwlock είναι καλύτερο (149.7 ms vs 159.4 ms, 6% βελτίωση).

Το rwlock επιτρέπει concurrent reads. Στο coarse-grained που ο παραλληλισμός είναι μηδενικός, τα παράλληλα queries παρέχουν βελτίωση. Στο fine-grained που ήδη υπάρχει υψηλός παραλληλισμός, το rwlock overhead (4-6 atomic ops vs 2 για mutex) είναι μεγαλύτερο από το όφελος. Επίσης τα queries είναι πολύ γρήγορα (απλό array read), οπότε το rwlock overhead δεν αντισταθμίζεται.

Επίδραση Query Percentage

Στο fine-grained mutex παρατηρείται μικρή διακύμανση (13.3-15.2 ms, 10-15%) ανεξαρτήτως query percentage. Το fine-grained ήδη επιτρέπει μέγιστο παραλληλισμό.

Στο coarse-grained mutex δεν υπάρχει επίδραση (159 ms σε όλα τα percentages) διότι το mutex δεν επιτρέπει concurrent reads.

Στο coarse-grained rwlock η επίδραση είναι minimal (0.1%) επειδή το query critical section είναι τόσο σύντομο που το rwlock overhead ισοσκελίζει το concurrent read benefit.

Πείραμα με usleep

Για να επιβεβαιωθεί ότι το rwlock υπερτερεί σε μεγάλα critical sections, προστέθηκε `usleep(10)` μέσα στο query operation. Με μεγαλύτερο query critical section, τα concurrent reads του rwlock υπερκαλύπτουν το overhead και το fine rwlock ξεπερνά το fine mutex.

Άρα το rwlock είναι ωφέλιμο μόνο όταν:
1. To read/write ratio είναι υψηλό (>75% reads)
2. To read critical section είναι μεγάλο (>10μs)
3. Υπάρχει υψηλή contention (coarse-grained ή λίγα accounts)

Συμπέρασμα

Το fine-grained mutex είναι η βέλτιστη επιλογή: 11.5x ταχύτερο από coarse-grained, απλούστερο από rwlock, και ανταγωνιστικό σε όλα τα query percentages.

Το granularity (fine vs coarse) είναι πολύ πιο σημαντικό από το synchronization mechanism (mutex vs rwlock). Το fine-grained με απλό mutex υπερτερεί του coarse-grained με rwlock.

ΑΣΚΗΣΗ 1.5

Περιγραφή προβλήματος

Διαφορετικές πολιτικές "barrier", ουσιαστικά όλα τα threads περιμένουν σε ένα κοινό σημείο συνάντησης και όταν φτάσουν όλα τότε προχωράνε. Υπάρχουν 4 (έγινε και μία επιπλέον προαιρετικά παρόμοια με την sense-reversal).

Περιγραφή λύσης

Σύμφωνα με την πολιτική

-**barrier**: Χρησιμοποιήθηκε έτοιμη η συνάρτηση από την standard βιβλιοθήκη pthread της C

-**condition variables**: Πολύ απλή υλοποίηση. Ουσιαστικά "μπαίνει" στον κοινόχρηστο χώρο, "λέει" ότι είναι εδώ και "πέφτει για ύπνο". Το τελευταίο thread ξυπνάει όλα και αρχικοποιεί την κοινή μεταβλητή. Όταν υπάρχει condition variable συνήθως χρησιμοποιείται while(συνθήκη){wait()} στην περίπτωσή μας δεν πρέπει να γίνεται αυτό και υπάρχει αντί αυτού if(συνθήκη){wait()} προκειμένου να μην μείνουν για πάντα κολλημένα.

-**my barrier (busy waiting)**: Υπάρχουν 2 μετρητές για το threshold, a και b, και έστω 4 συνολικά threads που πρέπει να συγχρονιστούν. Στην αρχή (init) a = 0 και b = 4. Έπειτα στον συγχρονισμό ακολουθείται αυτή η πολιτική:
- Όλα εκτός από το τελευταίο thread:
  - (όλα παρακάτω (a,b) με pointer)
  - (1) παίρνει το μικρότερο από (a,b) και κάνει ++
  - (2) πηγαίνει σε while όσο το μικρότερο από (a,b) είναι μικρότερο από threshold (busy waiting)
- Τελευταίο thread:
  - (όλα παρακάτω (a,b) με pointer)
  - (1) παίρνει το μεγαλύτερο από (a,b) και το κάνει 0
  - (2) παίρνει το μεγαλύτερο από (a,b) και κάνει ++
  - (3) δεν πηγαίνει στο while loop

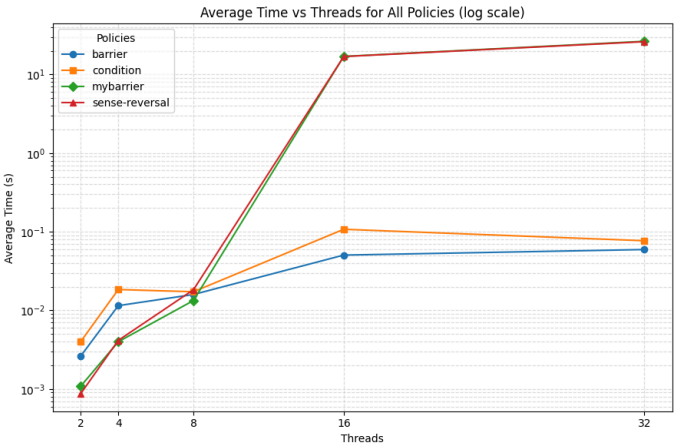
Η παραπάνω πολιτική είναι επαναχρησιμοποιήσιμη διότι διασφαλίζεται ότι όταν γίνεται set το μεγαλύτερο στο 0, δεν υπάρχουν άλλα threads που έχουν απομείνει από το προηγούμενο barrier. Εάν υπήρχαν τότε το (condition < threshold) δεν θα λειτουργούσε σωστά γιατί το condition θα πήγαινε από 4->0
- (4 < 4) false, άρα συνεχίζει (φεύγει από το "barrier")

- (0 < 4) true, άρα συνεχίζει να κάνει (busy waiting στο “barrier”)

-**sense-reversal (busy waiting)**: Χρησιμοποιεί παρόμοιο σκεπτικό με το δικό μας με τη διαφορά ότι αντιστρέφεται συνεχώς ένα flag.

Αποτελέσματα

Όσο αφορά το πώς επηρεάζεται ο χρόνος των 4 πολιτικών καθώς αυξάνονται τα threads με σταθερές επαναλήψεις



Για 2 threads. Base line policy “barrier”

Policy	Time [sec]	Speed up [times]
barrier	0.002626	1.00x
condition	0.004016	0.65x
mybarrier	0.001087	2.41x
sense-reversal	0.000874	3.01x

Για 4 threads. Base line policy “barrier”

Policy	Time [sec]	Speed up [times]
barrier	0.011486	1.00x
condition	0.018399	0.62x
mybarrier	0.003993	2.88x
sense-reversal	0.004152	2.77x

Για 8 threads. Base line policy “barrier”

Policy	Time [sec]	Speed up [times]
barrier	0.015906	1.00x
condition	0.017244	0.92x
mybarrier	0.013326	1.19x
sense-reversal	0.018285	0.87x

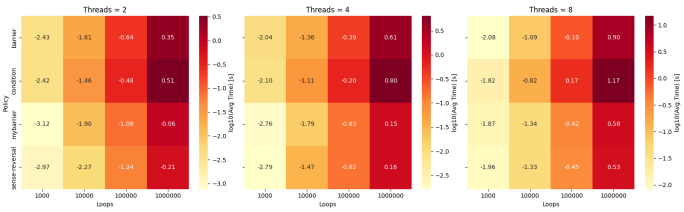
Για 16 threads. Base line policy “barrier”

Policy	Time [sec]	Speed up [times]
barrier	0.050443	1.00x
condition	0.107376	0.47x
mybarrier	16.940562	0.003x
sense-reversal	16.870212	0.003x

Για 32 threads. Base line policy “barrier”

Policy	Time [sec]	Speed up [times]
barrier	0.059315	1.00x
condition	0.076807	0.77x
mybarrier	26.364347	0.0022x
sense-reversal	26.067856	0.0023x

Όσο αφορά το πώς επηρεάζεται ο χρόνος των 4 πολιτικών καθώς αυξάνονται τα threads και οι επαναλήψεις



Συμπέρασμα-Παρατηρήσεις

Γενικά είναι αναμενόμενο ότι καθώς αυξάνονται τα loops για σταθερό αριθμό threads οι πολιτικές θα έχουν αντίστοιχη συμπεριφορά. Όπως παρατηρείται στα πειράματα, το condition variable είναι το πιο χρονοβόρο. Αυτό γίνεται διότι πέρα από τον χρόνο στην CPU “πληρώνεται” και ο χρόνος που χρειάζεται προκειμένου να φύγει το thread από την CPU και ο χρόνος που χρειάζεται για να ξαναμπεί όταν γίνεται **wait** και **broadcast**. Έπειτα ακολουθεί το **barrier**, θεωρητικά μια παραλλαγή του **condition variable**. Τέλος οι πιο γρήγορες είναι οι 2 υλοποιήσεις **busy waiting** που έχουν παρόμοιους ελάχιστους χρόνους. Αυτό γίνεται διότι δεν χάνεται καθόλου χρόνος σε “άσκοπες” αλλαγές κατάστασης μένοντας όλο το χρονομερίδιο που επιτρέπει ο **scheduler** πάνω στην CPU.

Τι γίνεται όταν ο αριθμός των νημάτων είναι μεγαλύτερος από αυτόν των πυρήνων

Γενικά μπορεί να παρατηρηθεί όπως είναι αναμενόμενο ότι όσο αυξάνεται ο αριθμός των νημάτων πέρα από το όριο που υπάρχει σε πυρήνες τόσο πιο πολύ ώρα χρειάζεται προκειμένου να επιτευχθεί ο συγχρονισμός. Αυτό γίνεται διότι ενώ κάποια νήματα είναι πάνω στην CPU και “περιμένουν” στο barrier κάποια άλλα δεν έχουν φτάσει καν λόγω του ότι δεν έχουν προλάβει να τρέξουν ακόμα πάνω στην CPU.

Πώς επηρεάζει πιο συγκεκριμένα η αύξηση των threads κάθε πολιτική

Για περισσότερα threads 16-32 μπορούν να παρατηρηθούν 2 διαφορετικές συμπεριφορές. Συγκεκριμένα τα **barrier** και **condition variable** έχουν παρόμοια συμπεριφορά εξαιτίας της σχετικά κοινής υλοποίησής τους. Πιο συγκεκριμένα το **condition variable** έχει θεαματικά καλύτερη απόδοση σε σχέση με τα 2 **busy waiting**. Αυτό γίνεται διότι μένουν πάνω στην CPU ακριβώς όσο χρόνο χρειάζονται με αποτέλεσμα να μπορούν να τρέξουν τα υπόλοιπα threads πάνω στην CPU που δεν έχουν ήδη τρέξει. Τα 2 **busy waiting**, **mybarrier** και **sense-reversal**, έχουν και αυτά μεταξύ τους την ίδια συμπεριφορά λόγω παρόμοιας υλοποίησης. Παρόλα αυτά παρατηρείται ότι είναι αντίθετη από αυτή των **barrier** και **condition variable**. Αυτό γίνεται διότι τα **busy waiting** είναι πολύ πιο “αλαζονικά”. Παρόλο που έχουν φτάσει στο **κοινό σημείο συνάντησης** δεν φεύγουν από την CPU μιας και προσπαθούν μανιωδώς, χωρίς αποτέλεσμα, να ελέγξουν αν μπορούν να φύγουν. Αυτή η “αλαζονεία” έχει σαν αποτέλεσμα να φεύγουν από την CPU όταν τους προεκτοπίζει ο **scheduler** προκειμένου να βάλει κάποιο άλλο thread πάνω στην CPU.